

obsidian-mcp-router — quick reference

v0.97.0 · multi-vault Obsidian MCP server, shipped by its Claude Code plugin — one install = everything (54 slash commands · 56 MCP tools · 49 skills)

github.com/tboome33/obsidian-mcp-router

The router in 30 seconds

A single MCP process that knows **all your Obsidian vaults** (local + remote). Each tool takes a `vault` parameter (or uses the default). No more registering one MCP server per vault.

- **Why HTTP, not the filesystem:** the router talks to a *live* Obsidian through its own REST API — never the raw markdown on disk. That gets you what only the running app can produce (Smart Connections' embeddings for `search_smart`, Templater's actual rendering for `execute_template`, `workspace.openLinkText` for click-to-open), and a write lands through Obsidian's own change pipeline instead of racing a human editing the same file live. The same HTTP call reaches a vault on a NAS or behind a WireGuard tunnel exactly like a local one — a filesystem tool can't do that without a network mount. The price: Obsidian has to be running, with one port + API key per vault to track (a filesystem-direct tool like `claude-obsidian` trades that away, at the cost of every plugin-computed capability above)
- **One plugin install = everything:** the Claude Code plugin ships *and launches* the server — declared in its root `.mcp.json` (key `router`, via `${CLAUDE_PLUGIN_ROOT}`). A manual `~/.claude.json` entry is the dev alternative; keeping both runs **two** server processes with duplicated tools (`mcp__obsidian-router__*` vs `mcp__plugin_obsidian-router_router__*`)
- **Local + remote vaults** treated identically — drop URL + API key into the config, done
- **Workspace→vault binding** (v0.90.0): which vault a cloned repository opens is no longer decided by its own `.env` — it's a binding confirmed once per machine, in *your* `config.json` (`confirm_workspace_binding`, or `setup-vault --attach`). A project's `.env` can only **propose** a vault now; the session-start briefing says what this workspace is bound to and how to change it
- **Cross-vault:** `vault: "*"` auto-fans-out across every vault
- **Lock mode:** restrict the session to a single vault (safety, focus) · **Auto-enrichment:** Claude suggests wiki saves at 3 natural moments, 4 modes (`ClaudeAsk` / `Hybrid` / `FullAuto` / `off`)
- **Conversion toolbox:** PDF / DOCX / XLSX / PPTX / image (OCR) / audio / YouTube / webpage / git repo → markdown; opt-in **Docling** high-fidelity PDF + `pdf_to_images` (the model sees a PDF page); **BM25 relevance filter** strips off-topic blocks before synthesis
- **Knowledge graph:** typed graph JSON + reading tours + neighbors + shortest link path; export as `llms.txt` or **OKF bundle** (Google's Open Knowledge Format v0.1, with one of the ecosystem's first validators)
- **Wiki analysis:** `find_boundary_pages` ranks heavily-linked-but-thin "frontier" pages; `find_twin_pages` finds quasi-duplicate pairs by cosine over the Smart Connections vectors, with a threshold derived from *your* vault's own distribution — and works on remote vaults too (bridge $\geq 0.9.0$ serves the vector store)

- **Click-to-open:** clickable links in chat via the bridge's `GET /open/<path>` route; `open_in_obsidian` does the same server-side (no browser — ideal for Claude Desktop). The link survives losing the vault's disk (`insecurePort` declarable in the config, exported only on explicit `--with-click-to-open`), and a one-click `GET /ping?v=<vault>` self-test proves *which* vault answers on a port
- **Vault-creation wizard:** `plan_vault` → adjust → `provision_vault` , defaults-first (happy path = 1 interaction)
- **Multi-tenant mode** (opt-in): scope an instance by env vars — vault whitelist, read-only, per-user write audit — for MCPHub/proxy deployments
- **11 hooks:** 3 come active with the plugin for every installer (hot-cache load + decisions recall + workspace briefing); the other 8 (session auto-journal, wiki autocommit, link linter, doc-drift checker, update check, hot-cache guard...) are opt-in via `node scripts/setup-vault.mjs --install-hooks`

How the pieces fit — the dependency chain

```
Obsidian ← Local REST API (community plugin) ← BRIDGE (mcp-router-bridge)
  ↑ HTTP per vault (port + apiKey from the Local REST API plugin)
MCP SERVER (obsidian-mcp-router) – Node ≥ 20.19.0 process on the PC
  ↑ MCP over stdio, spawned by Claude Code
CLAUDE CODE PLUGIN (obsidian-router) – commands + skills + agents + hooks
  · the plugin SHIPS and LAUNCHES the server itself
```

- **Bridge** — runs *inside* Obsidian; registers extra routes on Local REST API's HTTP server (`/search/smart` , `/templates/execute` , `/open/*` , `/ping` , `PUT /vault-cas/*` ≥ 0.7.0 — atomic `ifMatch` writes —, `GET /smart-env/sources` ≥ 0.9.0 — serves the Smart Connections vector store, a dot-directory the REST API itself won't serve, so `find_twin_pages` works on remote vaults —, presence heartbeat). **Optional per vault:** without it, core file CRUD still works (plain Local REST API routes), but smart search, Templater execution, click-to-open links, atomic conditional writes and remote twin-page analysis don't (`ifMatch` then degrades to a checked non-atomic fallback). Updates itself via BRAT from GitHub releases.
- **MCP server** — Node process spawned by Claude Code. Requires the **Local REST API** plugin in every vault (mandatory) and its registry at `~/.claude/obsidian-mcp-router/config.json` (maintained by `setup-vault.mjs`); the bridge is optional per vault (see above).
- **Claude Code plugin** — its commands and skills orchestrate the server's MCP tools; two hooks (hot-cache load, decisions recall) read vault files directly from disk.

The plugin ships the server — installing (or updating) the plugin installs (or updates) the server in the same move; fresh installs self-heal `node_modules` via a zero-dependency `bin/` bootstrapper. No `~/.claude.json` entry needed — that path remains for dev checkouts only (`claude --plugin-dir <checkout>` replaces the installed plugin for a session).

Vault setup

1. Have Node.js ≥ 20.19.0. Conversion backends are opt-in (no npm `postinstall`): `npm run install-markitdown` (needs Python 3.10+ on `PATH`), `npm run install-docling` — until installed, the

conversion tools stay listed and return an actionable install hint

2. Check whether that opt-in is done *before* a call fails: every `list_vaults` response carries `conversionToolbox` (`available` , `via` , `verified` , `hint`), and `meta-status` renders it in one line. **Eight** tools stop working without markdown; `youtube_to_markdown` falls back to yt-dlp captions (only where yt-dlp is itself installed), and `git_repo_to_markdown` (repomix) and `pdf_to_markdown_docling` are unaffected. Silence the offer for good with `OBSIDIAN_ROUTER_SKIP_MARKITDOWN=1` .
3. **Easiest path:** `/obsidian-router:meta-attach-vault` — interactive wizard that attaches a vault to a workspace (common case), bootstraps a standalone vault, or registers a remote one. Provisions plugins + scaffolds the wiki + binds `.env` + conventions picker.
4. CLI alternative:

```
npm run setup-vault -- "C:\VAULTS\MyVault"
```

⇒ installs/configures **Local REST API** + bridge plugins, writes `.env` , allocates a unique port, adds to the registry. The 3 base hooks come active with the plugin; the 8 extra hooks are opt-**in** via `node scripts/setup-vault.mjs --install-hooks` .
5. Optional for `search_smart` : **Smart Connections** + `obsidian-mcp-router-bridge` · for Templater: **Templater**
6. Verify: `/obsidian-router:meta-status` or just say "*diagnose the router*"

Configuration — the workspace `.env` (v0.87.0+: the **ONLY** keys the router takes from a repository's file — anything else in it is ignored and named on the router's stderr)

VAULT_PATH	Path of the current vault. Auto-detection: if it matches a registered vault, that vault becomes the default. Written by <code>setup-vault.mjs</code> .
OBSIDIAN_ROUTER_DEFAULT_VAULT	Explicit override of the default vault for this process — from the HOST only (your MCP server declaration, a launcher, your shell), where it wins over <code>VAULT_PATH</code> and <code>config.defaultVault</code> . The same variable in a project's <code>.env</code> is a proposal : reported by <code>list_vaults</code> as <code>bindingHint</code> and never applied, because a workspace is very often a cloned repository. What decides is the workspace's confirmed binding in your own <code>config.json</code> (<code>workspaceBindings</code>) — <code>confirm_workspace_binding</code> or <code>setup-vault --attach</code> records it, and the session-start briefing tells you which state you are in.
OBSIDIAN_ROUTER_LOCKED	Locks the router to one vault for the session. Any tool call to another vault throws. Set via <code>/obsidian-router:lock <vault> --persist</code> . Same origin rule : an authority from the host, a proposal from a project file — locking is the strongest way of choosing where writes land. <code>--persist</code> records <code>locked:</code>

`true` on the binding, which is what a restart reads; the `.env` line it also writes is a portable hint for another machine.

OBSIDIAN_ROUTER_AUTO_ENRICH

Wiki auto-enrichment mode: `ClaudeAsk` (default) | `Hybrid` | `FullAuto` | `off`. Set via `/obsidian-router:auto-mode <Mode> --persist`. **ONE value is refused from a workspace file (v0.89.0):** anything that canonicalises to `FullAuto` (`fullauto`, `full`, `full-auto`, `auto`, any casing) is not applied, because that mode is standing permission to write into a vault without asking and the `.env` a cloned repository carries must not grant it. (This closes the `.env` door only: a repository's `CLAUDE.md` can still ask Claude to call `set_auto_enrich_mode`, which the router cannot tell from your own call — the `auto-mode` skill tells Claude to set `FullAuto` on your request only, never on a file's instruction.) The key stays accepted and the other three modes still work from a file. `FullAuto` comes from the MCP host's server declaration or from a `set_auto_enrich_mode` call in the session; `--persist` refuses to write it and applies it to the session instead. A refusal is named on the router's stderr and carried by `list_vaults` as `autoEnrichModeRefused`.

MD_ALLOWED_PATHS • MD_SHARE_DIR

Conversion sandbox: the local paths the markdownify tools may read, and where they may write. Absent = no sandbox. The two names are one setting, and a workspace file may only NARROW it: its value is taken only when the host set neither and the instance is not gated (READONLY / ALLOWED_VAULTS / USER_ID) — otherwise it is withheld and named on stderr.

OBSIDIAN_ROUTER_NO_*

The 14 opt-outs enumerated in `src/helpers/workspace-dotenv.mjs`, accepted by name and never by shape (an unlisted `NO_*` is ignored): `NO_WATCH` (config hot-reload), `NO_HOT_CACHE_LOAD`, `NO_DECISIONS_RECALL`, `NO_WIKI_QUERY_FIRST`, `NO_WIKI_AUTOCOMMIT`, `NO_SESSION_JOURNAL`, `NO_HOT_CACHE_GUARD`, `NO_LINT_VAULT_LINKS`, `NO_DOC_STARTUP_CHECK`, `NO_DOC_PROPAGATION_CHECK`, `NO_UPDATE_CHECK`, `NO_AUTO_INSTALL_HOOKS`, `NO_AUTO_CONFORMANCE`, `NO_OKF_PROJECTIONS`. Each switches a convenience off, none a guard; all accept `1` / `true` / `yes` / `on`.

Configuration — host / shell variables (never taken from a workspace `.env`: set them in the MCP host's server declaration, in the launcher of a served instance, or in the shell)

OBSIDIAN_ROUTER_ALLOWED_VAULTS

Multi-tenant: comma-separated whitelist of vaults this instance sees. Others are skipped.

OBSIDIAN_ROUTER_READONLY	Multi-tenant: truthy (<code>1</code> / <code>true</code> / <code>yes</code> / <code>on</code>) hides the 18 write tools — <code>write_file</code> , <code>append_to_file</code> , <code>patch_file</code> , <code>set_frontmatter</code> , <code>merge_frontmatter</code> , <code>move_file</code> , <code>delete_file</code> , <code>execute_template</code> , <code>download_page_assets</code> , <code>pptx_extract_assets</code> , <code>build_wiki_graph</code> , <code>provision_vault</code> , <code>register_remote_vault</code> , <code>write_bundle</code> , <code>record_source</code> , <code>install_conventions</code> , <code>refresh_okf_projections</code> , <code>build_search_index</code> — filtered from ListTools AND refused at call time.
OBSIDIAN_ROUTER_USER_ID	Multi-tenant: audit trail — every successful write appends an attributed line to the vault's <code>wiki-meta/journal.md</code> .
VAULT_<NAME>	A whole vault defined in one env var (JSON: <code>name</code> , <code>baseUrl</code> , <code>apiKey</code> ...) — dashboard-editable on MCPHub deployments.
OBSIDIAN_ROUTER_CONFIG · MARKITDOWN_PATH · DOCLING_PATH · REPOMIX_PATH · YTDLP_PATH · HTTPS_PROXY · HF_HOME...	Config path and tool overrides: read from the host's environment only (README § environment). A spawned tool receives a named allowlist of variables, never the router's whole environment (v0.87.0).

Important files — where things live

~/.claude/obsidian-mcp-router/config.json	Main router config (vaults, port registry, defaultVault, disabledVaults, remoteVaults). Hot-reload enabled.
~/.claude.json	Manual/dev setup only: a hand-registered server entry pointing to a checkout. The plugin declares its server in its own root <code>.mcp.json</code> (key <code>router</code> , <code>\${CLAUDE_PLUGIN_ROOT}</code>); a hand-registered entry <i>plus</i> the plugin = two processes with duplicated tools (dev machines may keep both deliberately).
~/.claude/settings.json	The 8 opt-in hooks land here via <code>setup-vault.mjs --install-hooks</code> (the 3 base hooks ship active in the plugin's <code>hooks/hooks.json</code> — 11 hooks total). The plugin itself is enabled per-workspace via <code><workspace>/ .claude/settings.json</code> — 54 commands + 49 skills ≈ 10k context tokens, only load them where you use Obsidian.
<workspace>/ .env	Auto-loaded at router boot — but only the workspace keys above are taken, and the parent environment always wins; every other key is ignored and named once on the router's stderr, and the keys applied are named too (v0.87.0+).

<code><vault>/CLAUDE.md</code>	Vault wiki consigne — loaded by Claude Code into context. Includes the auto-enrichment consigne (4 modes) + installed conventions.
<code><vault>/wiki-meta/{catalog,journal,hot,overview}.md</code>	Karpathy LLM-wiki scaffolds (catalog, append-only journal, hot cache, overview) — under <code>wiki-meta/</code> (user pages live under <code>wiki/</code>). Named <code>catalog / journal</code> rather than <code>index / log</code> because OKF reserves those two basenames; the old names are still read. Scaffolded by <code>/obsidian-router:wiki</code> .
<code><vault>/wiki-meta/Sessions/</code>	Per-session detail journals (auto-written by the <code>session-auto-journal</code> hook; <code>journal.md</code> stays a thin index).
<code>obsidian-mcp-router/docs/</code>	<code>auto-enrichment.md</code> (4 modes + 4 placement channels), <code>features/</code> (13 prose feature pages), <code>architecture.md</code> , this PDF (EN + FR).

54 slash commands /obsidian-router:<name> · auto-trigger on natural language

(EN + FR)

 17 MCP wrappers — one per core vault tool

Categories: **discover** · **read** · **write** · **manage** · **template** · **convert**

COMMAND	EFFECT	TRIGGER PHRASE (EN/FR)
discover-list-vaults	List vaults (online/latency/lock state)	"list my vaults" / "liste mes vaults"
discover-list-files	List files in a vault directory	"list files in Sessions" / "liste les fichiers de X"
read-get	Read a file (markdown + frontmatter)	"show me X" / "montre-moi X"
read-search	Plain-text (substring) search	"grep for <text>" / "trouve <texte>"
read-search-smart	Semantic search (Smart Connections)	"find notes about X" / "trouve mes notes sur X"
read-frontmatter	Read frontmatter (object or one key)	"metadata of X" / "quel est le statut de X"
write-create-or-replace	PUT — create or replace a file	"save this as X.md" / "crée une note X"
write-append	POST — append to a file	"append to my journal" / "ajoute à X"
write-patch	Surgical PATCH (heading/block/frontmatter)	"edit X section in Y" / "édite la section X dans Y"
write-frontmatter-set	Set one frontmatter key	"tag this with X" / "passe le statut de X à closed"
write-frontmatter-merge	Set multiple keys in sequence	"on X set status=closed outcome=tp1"
write-bundle	Multi-file bundle — all-or-nothing apply with journaled rollback	"write these 4 pages atomically" / "écris ces pages d'un bloc"
manage-move	Move or rename a file	"move X to Y" / "renomme X en Y"

manage-delete	Delete a file (2-step confirm guard)	"delete X" then "yes confirm=true"
template-execute	Execute a Templater template	"run X template" / "rends Templates/X.md avec arg=v"
pdf-to-markdown	Local PDF → markdown (MarkItDown, fast plain-text)	"convert this PDF to markdown" / "convertis ce PDF"
pdf-to-markdown-docling	High-fidelity PDF → markdown (Docling, tables/layout, opt-in)	"convert with docling" / "conversion haute fidélité"

4 router-state commands (lock + auto-enrichment + port base)

COMMAND	EFFECT	TRIGGER PHRASE (EN/FR)
lock	Restrict the session to one vault (volatile or <code>--persist</code>)	"I only want to work on X" / "verrouille sur X"
unlock	Lift the lock, restore multi-vault routing	"unlock vaults" / "déverrouille les vaults"
auto-mode	Set the auto-enrichment mode (ClaudeAsk / Hybrid / FullAuto / off)	"switch to Hybrid mode" / "passe en mode Hybrid"
force-new-port-start	Draw a new allocation base for future vaults only — sealed two-phase plan, zero existing ports touched	"draw a new port base" / "tire une nouvelle base de ports"

2 workspace-binding commands (which vault this project writes to)

COMMAND	EFFECT	TRIGGER PHRASE (EN/FR)
bind-workspace	Wizard — bind this workspace to a PRIMARY vault, then optionally secondaries with a write tier each; also names an unanswered proposal from the project's dotenv file, and records a "no" as a refusal	"which vault is this project attached to" / "rattache ce workspace à un vault"
configure-secondary-vaults	Same wizard entered at its SECONDARY step — declare the secondaries and pick each write tier: read-only strict / on request / read-write	"add a secondary vault" / "mets X en lecture seule pour ce projet"

7 conversational helpers (setup + diagnostic)

COMMAND	EFFECT	TRIGGER PHRASE (EN/FR)
meta-setup	Manual/dev install (clone + npm link + <code>~/.claude.json</code> entry) — normal installs get the server from the plugin	"setup obsidian-mcp-router" / "installe le router"
meta-attach-vault	Wizard — attach a vault to a workspace (common), standalone, or remote	"attach a vault to this workspace" / "connecte mon vault distant"
meta-status	Health-check every vault with fix hints	"are my vaults reachable" / "diagnostique le router"
meta-sync-template	Propagate the reference vault's plugins/snippets/docs to vaults (picker)	"sync the template to all vaults" / "synchronise le template"
sync-from-github	Update one vault or the whole fleet straight from the GitHub skeleton (plugins, themes, snippets, docs) — no local dev repo needed	"sync my vaults from github" / "mets à jour depuis GitHub"
meta-audit-bridge-readiness	Audit click-to-open readiness (bridge, REST API, HTTP, live /open probe)	"is click-to-open ready" / "audite le bridge"
conventions	Install/remove/status/propagate CLAUDE.md conventions cross-vaults	"install source-type on X" / "installe la convention X"

24 knowledge-management commands (Karpathy-style LLM-wiki)

COMMAND	EFFECT	TRIGGER PHRASE (EN/FR)
wiki	Scaffold the wiki in a vault (index, log, hot, overview)	"set up a wiki" / "scaffold un wiki"
wiki-ingest	Ingest a source → entity/concept pages + cross-refs	"absorb this article" / "ingère cette URL"
wiki-query	3-tier RAG over the wiki (no web)	"what does my wiki say about X" / "d'après mes notes ..."
wiki-lint	Health check (orphans, dead links, index drift)	"audit my wiki" / "lint le wiki"
wiki-fold	Idempotent log rollup under wiki/folds/	"fold the log" / "compacte le journal"
hot-compact	Compact an oversized hot.md back to its cache contract (verified backup first)	"compact the hot cache" / "compacte le hot"
save	File the conversation as a typed wiki note (session/decision/ADR/...)	"save this" / "sauvegarde ça"
decision-consolidate	Compress a settled decision page to its essentials; move the full deliberation history to a verified archive note	"consolidate this decision" / "consolide cette décision"
autoresearch	Autonomous web→synth→file loop	"research X online" / "fais une recherche web sur X"
canvas	Create/edit Obsidian .canvas files	"create a canvas for X" / "crée un canvas pour X"
defuddle	Strip noise from web pages before ingestion	"defuddle <url>" / "nettoie cette page"
obsidian-bases	Create/edit Obsidian .base files (database views)	"create a base for X" / "crée une base pour X"
wiki-graph	Build the typed knowledge-graph JSON → <code>wiki-meta/graph/</code> + an <code>.understand-anything/</code> copy for the Understand-Anything dashboard (deterministic, no LLM)	"build the wiki graph" / "construis le graphe"

wiki-tour	Ordered pedagogical reading tour from the link topology	"give me a tour of this vault" / "par où je commence"
wiki-neighbors	One page's neighbors — links out, backlinks, or both	"what links to X" / "voisins de X"
wiki-path	Shortest link chain between two pages	"how is X connected to Y" / "quel rapport entre X et Y"
wiki-export	Export the vault as llms.txt / llms-full.txt or an OKF bundle	"export the wiki as llms.txt" / "exporte le wiki"
okf-export	Export a wiki subset as a shareable OKF v0.1 bundle (conformance self-checked)	"export this folder as an OKF bundle" / "publie en bundle OKF"
okf-projections	Regenerate the generated OKF navigation in wiki/ (root + per-directory index.md, newest-first log.md); auto-refreshed after writes; --check = drift report	"refresh the OKF projections" / "régénère les index du wiki"
okf-check	Validate any OKF bundle against the v0.1 conformance rules	"validate this OKF bundle" / "ce bundle est-il conforme ?"
wiki-refresh-digests	Regenerate per-page digest sidecars (concepts/claims/keywords)	"refresh the digests" / "rafraîchis les digests"
build-search-index	Build/refresh the local BM25 search index (plugin-free search tier, idempotent)	"build the search index" / "construis l'index de recherche"
wiki-boundary	Rank heavily-linked-but-thin "frontier" pages worth writing next	"what should I write next" / "pages frontière du wiki"
who-is-speaking	Identify the family member in a shared vault, lock routing per member	"who is speaking" / "c'est Karine"

The 56 MCP tools — what Claude actually calls under the hood

`list_vaults` · `list_files`

Discovery. Vault catalog with online status + latency (call first); directory listing.

`get_file` · `search` · `search_smart` ·
`get_frontmatter`

Reads. Full file (returns `contentSha256` — the `ifMatch` token); substring search; semantic search (Smart Connections embeddings, cosine scores); typed frontmatter. `vault: "*" = cross-vault fan-out`. **`search_smart` tells you two things about its own answer.** `freshness` flags each hit whose page has been modified since it was indexed (`changed` / `touched`) or has gone (`page-missing`) — open the page before quoting such a hit; a vault with no disk here answers `checkable: false` rather than implying currency. `folderExclusion` reports what was left out: **by default** `wiki-meta/Sessions` (chronological logs, 41.6% of the indexed pages on this fleet). Pass `excludeFolders: []` to get them back.

`write_file` · `append_to_file` · `patch_file` ·
`delete_file` · `set_frontmatter` ·
`merge_frontmatter` · `move_file`

Writes & file management. Create/replace; append; surgical patch by heading/block/frontmatter; guarded delete (`confirm: true`); one or many frontmatter keys; move/rename. `ifMatch` : replay `get_file` 's `contentSha256` and the write is refused (409) if the file changed since you read it — atomic with bridge $\geq 0.7.0$, checked fallback otherwise.

`execute_template`

Templater. Render a template, optionally write the result; args via `tp.mcpTools.prompt("key")` .

`lock_vault` · `unlock_vaults` ·
`set_auto_enrich_mode` ·
`confirm_workspace_binding` ·
`set_secondary_vault_mode`

Router state & workspace binding. Single-vault isolation; auto-enrichment mode switch; confirm, refuse or clear this workspace's vault binding (`{ vault }` , `{ vault, also: [ ... ] }` for several, `{ refuse: true }` to say no to a proposal, `{ clear: true }` for all vaults) — recorded in your own `config.json` , never in the project. `set_secondary_vault_mode` sets a secondary's **write tier**: `locked` (server-side refusal, no override), `soft` (the default — refused until confirmed once with `confirmSecondaryWrite`), `writable` . On a gated deployment (`READONLY` / `ALLOWED_VAULTS` / `USER_ID`) both are refused: one directory serves every caller, so a persisted answer would speak for all of them.

plan_vault · provision_vault · register_remote_vault	Vault provisioning. Read-only plan (defaults + questionnaire + warnings) → one-call creation (plugins, port, API key, wiki scaffold, registry). Local-only. <code>register_remote_vault</code> records an already-running remote vault (name, <code>baseUrl</code>, <code>apiKey</code>) straight from a conversation — always into <code>config.json</code>, never into a project's dotenv file, and hidden on gated deployments.
pdf/docx/xlsx/pptx/image/audio_to_markdown	Local conversion (Markdown — opt-in via <code>npm run install-markitdown</code>; the tools stay listed and return an install hint when the backend is missing). Office/PDF/image OCR/audio transcription → markdown text; chain with <code>write_file</code>.
pdf_to_markdown_docling · pdf_to_images	High-fidelity PDF (opt-in Docling): layout + table-structure recognition. <code>pdf_to_images</code> renders pages to PNG so the model can see them (capped pages/bytes).
pptx_extract_assets	Deck images to disk, no Python (a PPTX is a ZIP): each embedded image written once, with the slide positions that show it (short of any part reported in <code>skipped</code>) — the bytes <code>pptx_to_markdown</code> cannot return. A <i>write tool</i>: an <code>outdir</code> inside a vault meets that vault's tier, and <code>createOnly: true</code> on a shared one; outside every vault, only the temp directory is allowed.
youtube/bing_search/webpage_to_markdown · git_repo_to_markdown	Remote conversion. URL → markdown (SSRF-guarded); YouTube transcripts; git repo → single markdown bundle (repomix, optional Tree-sitter compression). <code>webpage_to_markdown</code> accepts <code>relevanceQuery</code> (BM25) and <code>citations: true</code> — inline links move into numbered footnotes with a reference list, one per destination, leaving code, images and wikilinks alone. With both, the filter runs first, so markers and definitions match one-to-one.
filter_relevant_blocks	Relevance filter. Deterministic BM25 second pass over markdown you already have — drops off-topic blocks, keeps frontmatter/headings, multiple no-op safety nets. No fetch, no LLM.
extract_page_metadata · propose_linked_sources · download_page_assets	Web ingestion support. Deterministic page metadata (JSON-LD/OpenGraph); scored link-following candidates; image download into the vault.
get_wiki_context_pack · build_wiki_graph · build_wiki_tour · get_page_neighbors · wiki_path	Context & graph. Structured context envelope for external agents; typed knowledge graph; reading tours; per-page neighbors; shortest link path. Every item of the context pack carries its <code>source</code> — <code>index</code> (the vault's

own catalog) and `graph` (a wikilink) are **navigation, authoritative**; `semantic` is augmentation and never the sole support for a factual claim. A pack with no navigational anchor says so: `answer-relies-on-semantic-only`.

`find_boundary_pages` · `find_twin_pages`

Wiki analysis. Frontier pages worth writing next (linked a lot, say little); quasi-twin pairs by cosine over the Smart Connections vectors, threshold derived from the vault's own distribution, every pair carrying the evidence to dismiss it. Both answer `available: false` with a reason (never a fake "0 found") when they cannot look. `find_twin_pages` works on **remote vaults** too (bridge $\geq 0.9.0$ serves the vector store).

`write_bundle` · `refresh_okf_projections` · `build_search_index`

Wiki maintenance. Journaled multi-file bundle — all-or-nothing apply with rollback; regenerate the OKF navigation (per-directory `index.md` + newest-first `log.md` — generated files, never edit by hand); local BM25 search index (works on every vault, no plugin needed).

`record_source` · `audit_sources`

Source ledger. Record where ingested content came from (URL, hash, date); audit a vault's sources for drift and gaps.

`audit_vault_conventions`

Conventions audit. Which conventions a vault is really under: the file the router resolves (or the two it cannot choose between), the recommended ones missing, template copies and inherited backups — each finding with a repair to approve, never applied on its own.

`install_conventions`

Conventions installer. Install library conventions by id into the vault's conventions file in ONE compare-and-swap write (texts read from the package, never pasted), then read the file back and verify; `languages` takes the vault's own value, a retired convention is reported and never written, `dryRun` returns the picker's state.

`build_open_link` · `open_in_obsidian` · `get_view_link`

Navigation. Ready-to-paste click-to-open links (path-verified against the vault; on a diskless vault the link is still built, flagged `pathVerified: false`); server-side open (raise Obsidian window, optional heading anchor) — no browser round-trip; ephemeral browser view-link to a live GUI for remote deployments.

Tips · Type `/obsidian-router:` in Claude Code → autocomplete · Every command accepts a `vault` argument (wrappers fall back to the default) · `vault: "*"` on `search` / `search_smart` = cross-vault fan-out · Trigger

phrases are indicative — Claude recognizes variants · Write results carry a ready-made `clickToOpenUrl` — one click opens the note in Obsidian · Full docs: `README.md` , `docs/features/` (13 pages), `docs/auto-enrichment.md` in the repo.